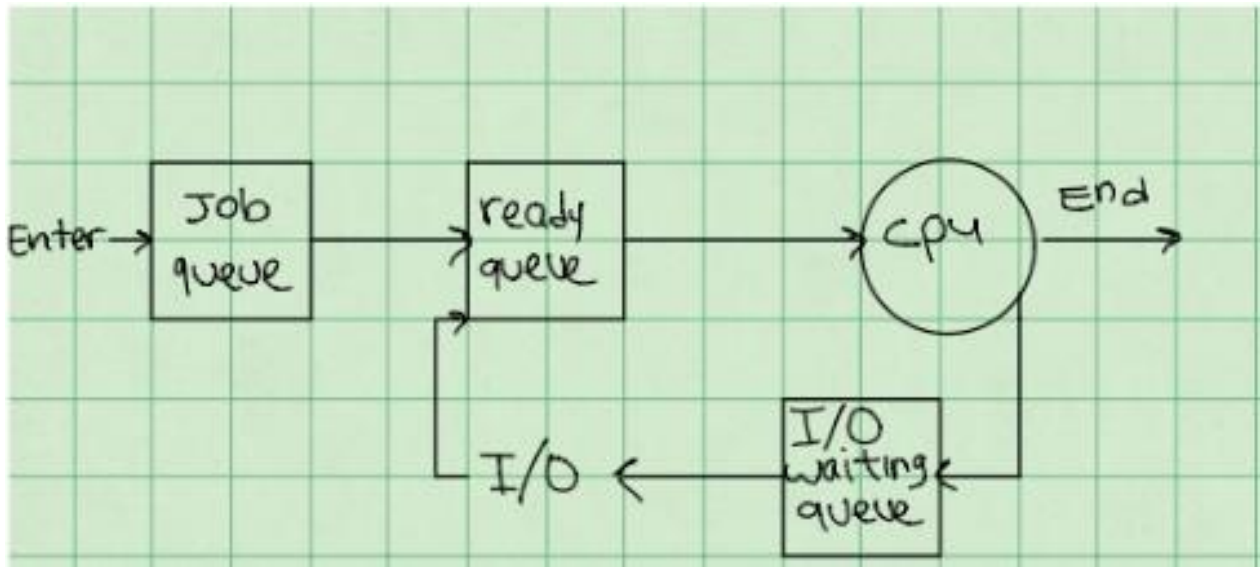


Scheduling algorithms

תזמון



- תהליך נכנס למערכת – מוכנס לתור job queue – תור זה יכיל את כל התהליכים במערכת שרוצים להיכנס למצב ריצה.
- תהליכים המוכנים לריצה נכנסים לתור ready queue – תור הזיכרון. תהליכים נכנסים לזיכרון והם זמינים לריצה.
- i/o waiting queue – תהליכים בהמתנה. ברגע שמתקבלת פעולת קלט/פלט הדרושה, התהליך חוזר לתור ready.

● המרכיב העיקרי של כל מערכת הפעלה, שמבצע את פעולת התזמון נקרא מתזמן

מתזמנים - schedulers

- כל עוד תהליך קיים הוא עובר בין מספרי תורי תזמון. מערכת הפעלה בוחרת תהליכים מתוך התורים ומשתמשת ב 2 מתזמנים :
- long term scheduler – מתזמן את התור job queue ומחליט אילו תהליכים יעברו למצב ready.
- cpu scheduler – נקרא גם short term scheduler , המתזמן את התור ready queue ומחליט מי מהתהליכים יקבל את ה cpu.
- medium-term scheduler - הוא רכיב שנמצא בין short-term scheduler לבין long-term scheduler . המטרה של מתזמן זה היא לנהל את זיכרון המערכת ולבצע החלפת תהליכים בין הזיכרון הראשי RAM לבין זיכרון המשנה secondary storage.

מתזמנים - schedulers

- החלטות המתזמן קורות בנסיבות הבאות:

1. תהליך עובר ממצב ריצה למצב המתנה.
2. תהליך עובר ממצב ריצה למצב ready.
3. תהליך עובר ממצב המתנה למצב ready.
4. תהליך נגמר.

- רוב התהליכים מתחלקים ל 2 קבוצות –

1. i/o bound process – תהליך הזקוק כל הזמן לקלט/פלט ונמצא הרבה במצב המתנה.
2. cpu bound process – תהליך צריך כל הזמן את פעולת ה cpu.

קריטריונים לתזמון

כאשר מחליטים באיזה אלגוריתם להשתמש ובאיזה מצב, ניקח בחשבון את הקריטריונים הבאים:

1. ניצול cpu – נוודא שהcpu כל הזמן מנוצל ועובד.
2. לכל תהליך יהא חלק שווה בתוך CPU.
3. תפוקה – יעילות הרצת תהליכים חופפים (מספר תהליכים שסיימו לרוץ ביחידת זמן אחת).
4. זמן סבב turn around – סכום הזמן שהתהליך בזבז בהמתנה לזיכרון, המתנה בתור הready, ריצה בcpu והמתנה ל i/o.
5. זמן המתנה – הזמן הכולל בו התהליך העביר בready. (לא נחשב את הזמן בו המתין ב i/o queue).

• מטרה –

1. ניצול cpu מקסימלי.
2. מזעור זמן סבב, זמן המתנה וזמן תגובה.

FCFS – FIRST COME FIRST SERVED

- (1) FCFS - First come first served - ראשון בא ראשון ישרת.
- ה-CPU מוקצה לתהליך הראשון שביקש אותו.
 - מיוחס בעזרת תור FIFO.
 - ברגע שתהליך נכנס לready queue . ברגע שה-CPU פנוי הוא מוקצה לתהליך הראשון בתור.

מאפיינים:

- יישום פשוט, מהיר ולא מורכב.
- אינו מאפשר מתן זכות קדימה לתהליך.

FCFS – FIRST COME FIRST SERVED

<u>Process</u>	<u>burst time</u> = כניסות
P ₁	24 ↑ 1ms CPU
P ₂	3
P ₃	3

סדר הגעה: P₁, P₂, P₃ . זמן המתנה: 0 - P₁ (ראשון)
 P₂ - 24 (מתנה P₁)
 P₃ - 27 (מתנה P₁, P₂)

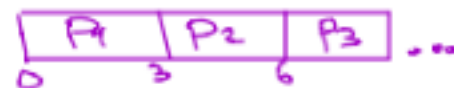


זמן המתנה
ארוך

$$\frac{0 + 24 + 27}{3} = 17 = \text{זמן המתנה ממוצע}$$

FCFS – FIRST COME FIRST SERVED

סדר העדפה: P_2, P_3, P_1 $1ms$ (המתנה) =
 P_2 (0 - 3) P_3 (3 - 6) P_1 (6 - 9)
 (מתנה 3) (מתנה 3) (מתנה 3)
 (מתנה 6) (מתנה 3) (מתנה 3)



אזכור: הנתונים
 הנתונים הם לא
 (הנתונים הם לא)

$$\frac{0 + 3 + 6}{3} = 3 \quad \text{ms (מתנה):}$$

Process	burst time: ms
P_1	2
P_2	3
P_3	3

↑
 ms
 CPU


```

//fcfs
#include<stdio.h>
main()
{
//wt=wait time,tat=time turnaround
int  wt[20],tat[20],i, k, n=3, temp;
float wtavg, tatavg;
int  p[] = {0,1,2}; //processes
int  bt[] = {24,3,3}; // burst time
wtavg = wt[0] = 0; // wt for first process
tatavg = tat[0] = bt[0];

for(i=1;i<n;i++)
{
    wt[i] = wt[i-1] + bt[i-1]; //wait time current proc = cpu time + wait time (prev proc)
    tat[i] = tat[i-1] + bt[i]; //time turnaround = tat prev + current burst time
    wtavg = wtavg + wt[i];
    tatavg = tatavg + tat[i];
}
printf("\nPROCESS\t\tBURST TIME\tWAITING TIME\tTURNAROUND TIME");
for(i=0;i<n;i++)
    printf("\n%d \t\t %d \t\t %d \t\t %d \t\t ",p[i],bt[i],wt[i],tat[i]);
printf("\nAverage Waiting Time is --- %f",wtavg/n);
printf("\nAverage Turnaround Time is --- %f",tatavg/n);
}

```

PROCESS	BURST TIME	WAITING TIME	TURNAROUND TIME
0	24	0	24
1	3	24	27
2	3	27	30
Average Waiting Time is --- 17.000000			
Average Turnaround Time is --- 27.000000			

FCFS – FIRST COME FIRST SERVED

- עשוי להיווצר מצב של convoy effect - אפקט השיירה - כאשר כל התהליכים ממתינים לתהליך אחד ארוך שיסיים עם ה CPU.
- ברגע שה CPU מוקצה לתהליך, התהליך מחזיק בו עד לרגע שבו הוא משחרר אותו.
- באלגוריתם מסוג זה תיתכן הרעבה, ברגע שתהליך שקיבל את ה CPU ונכנס ללולאה אינסופית.

SJF – shortest job first

מאפיינים:

- התהליך עם זמן הביצוע הקצר ביותר קיבל את העדיפות הגבוהה ביותר במעבד.
- ללא יכולת מתן זכות קדימה.

• התהליך בעל זמן CPU הנקיצר (burst time) יקבל את ה-CPU.

• בעידיה ויש 2 תהליכים עם זמן CPU שווה $\leftarrow$ יופעל אלגוריתם FCFS למחזור התהליך הקודם יותר.

• שאלה - כמה זמן המתנה ממוצע

במקרה זה היה אם האלגוריתם תזמון

היה FCFS?

שאלה: process burst time SJF יתאמן את

ותהליכים בסדר

(הכאן) P_1, P_2, P_3, P_4

P_4 - 5 ממוטן, P_1 - 3 ממוטן

P_3 - 9 ממוטן, P_2 - 16 ממוטן

process	burst time
P_1	6
P_2	8
P_3	7
P_4	3

$$3 + 9 + 16 / 4 = 7 \quad \text{SJF} \quad \text{זמן המתנה ממוצע}$$

SJF – shortest job first

- תרגיל – כתוב תכנית בשפת C המממשת את אלגוריתם SJF
- הנחייה – יש למיין את התהליכים לפי סדר עולה של ה burst time ולאחר מכן להפעיל את אלגוריתם fcfs

SJF – shortest job first

- פתרון – נוסיף את הקטע קוד מיון הבא לפני חישוב כל הזמני המתנה

```
for(i=0;i<n;i++)
    for(k=i+1;k<n;k++)
        if(bt[i]>bt[k])
        {
            temp=p[i]; // sort processes array
            p[i]=p[k];
            p[k]=temp;

            temp=bt[i]; // sort burst time array
            bt[i]=bt[k];
            bt[k]=temp;
        }
```

PROCESS	BURST TIME	WAITING TIME	TURNAROUND TIME
3	3	0	3
0	6	3	9
2	7	9	16
1	8	16	24
Average Waiting Time is --- 7.000000			
Average Turnaround Time is --- 13.000000			

Priority scheduling - non preemptive

- כל תהליך מקבל עדיפות ומתזמן התהליכים בוחר את התהליך בעל העדיפות הגבוהה ביותר ראשון לריצה.

• אלג' נותן לכל תהליך עדיפות.
• תהליך בעל העדיפות הגבוהה ביותר מקבל את ה-CPU.
• אם 2-5 תהליכים עדיפות צריכים שימוש - FCFS.
• **אלג' / SJF** וזו אלג' עדיפות כאשר העדיפות היא **זמן איזני**.
• אחסון - עיני לגרום לתהליכים עם העדיפות נמוכה לחכות קצת.
• פתרון: aging - שיטה בה מעדיפים עדיפות של תהליך בכל
שמן נמתנה שזו קצת.

Priority scheduling - non preemptive

• דוגמא

תכנותן ותהליכים	Process	Burst time	Priority	※: אמצעים
יחידה קבוצה :	p0	10	3	
	p1	1	1	
p1, p4, p0, p2, p3	p2	2	4	
	p3	1	5	
	p4	5	2	

• שאלה – מה יהיה זמן המתנה ממוצע בדוגמא הנ"ל ?

Priority scheduling - non preemptive

• תשובה –

• P1 – לא ממתין

• P4 – ממתין 1

• P0 – ממתין $6=1+5$

• P2 – ממתין $16=6+10$

• P3 – ממתין $18=16+2$

• סה"כ $= 5/1+6+16+18$

• זמן המתנה $= 8.2$

תכנון תהליכים
עודף קסדר :

P_1, P_4, P_0, P_2, P_3

Process	Burst time	Priority	סדר קסדר :
P0	10	3	
P1	1	1	
P2	2	4	
P3	1	5	
P4	5	2	

Priority scheduling - non preemptive

- תרגיל – כתוב תכנית בשפת C המממשת את אלגוריתם Priority
- הנחייה – יש להוסיף מערך של עדיפויות.
- למיין את התהליכים לפי סדר העדיפויות.
- להפעיל את אלגוריתם fcfs.

```

for(i=0;i<n;i++)
    for(k=i+1;k<n;k++)
        if(pri[i] > pri[k])
        {
            temp=p[i]; //sort processes
            p[i]=p[k];
            p[k]=temp;

            temp=bt[i]; // sort burst time arr
            bt[i]=bt[k];
            bt[k]=temp;

            temp=pri[i]; // sort priority arr
            pri[i]=pri[k];
            pri[k]=temp;
        }

```

Priority scheduling - non preemptive

PROCESS	PRIORITY	BURST TIME	WAITING TIME	TURNAROUND TIME
1	1	1	0	1
4	2	5	1	6
0	3	10	6	16
2	4	2	16	18
3	5	1	18	19
Average Waiting Time is --- 8.200000				
Average Turnaround Time is --- 12.000000				

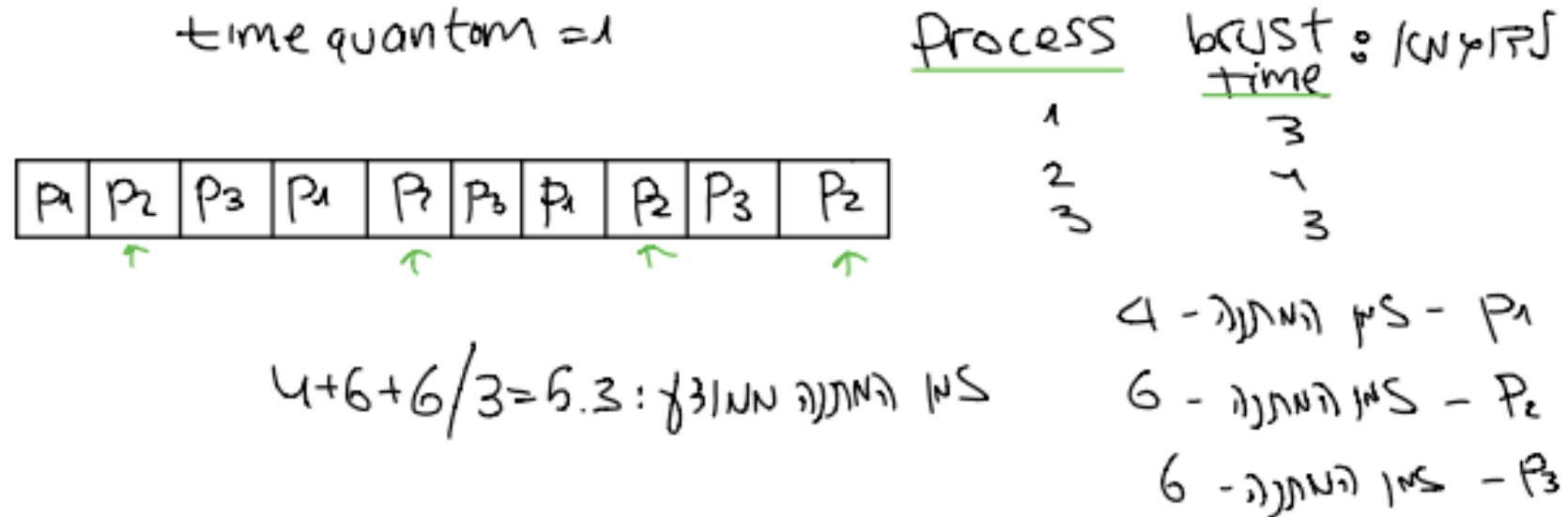
Round Robin

מאפיינים:

- אפקטיבי בסביבה בה מתקיימת חלוקת זמנים

- התהליכים מתחלפים ביניהם תוך כדי ריצה
- מאצרת יחידת זמן t time quantum
- תור ה- ready הופך לתור "מאצלי"
- המתבצעת מקצה קבל פעם להתחיל אחר אחר ה- cpu & time quantum.

Round Robin



* איתרון - נבחר מתחילים קצרים מחד
 * חיסרון - תחילים הקצרים במעט מה time quantum - מיותר לעצור אותם כשהם לבואת סיום

מטלה

- יש לכתוב תכנית המממשת את אלגוריתם Round Robin
- פלט לדוגמא -

PROCESS	BURST TIME	WAITING TIME	TURNAROUND TIME
1	3	4	7
2	4	6	10
3	3	6	9

The Average Waiting time is -- 5.333333
The Average Turnaround time is -- 8.666667

פתרון מטלה

• הקוד נמצא במודל rountrobin.c